ENGINE ONBOARDING · CRATE DEFUSE · RUST 2024 · NO DEPENDENCIES · #![FORBID(UNSAFE_CODE)] · WASM32

An incremental, parallel, field-level def-use evaluator

How the engine binds, schedules, speculates and re-evaluates, and how to work on it without breaking the four rules

src engine.rs 5670 · ctx.rs 761 · obs.rs 519 · value.rs 474 · tree.rs 453 · sched.rs 226 · hash.rs 102 lines

ex toy language 2172 · interpreter 686 · trace replay 1546 · typing · chains · fuzz · shrink · 72 tests

ROADMAP

Ten stops, from the model to the open problems

1
The model
four rules

2
Data
nodes, order treap, indexes, values

3
The loop
evaluate → rounds → drain

4
Binding
`walk` and pending structure

5
Client features
kinds, contributions, folds, claims, env

6
Incremental
edits, epochs, cutoff, regions

7
Harness
toy, interpreter, tests, fuzz, benches, traces

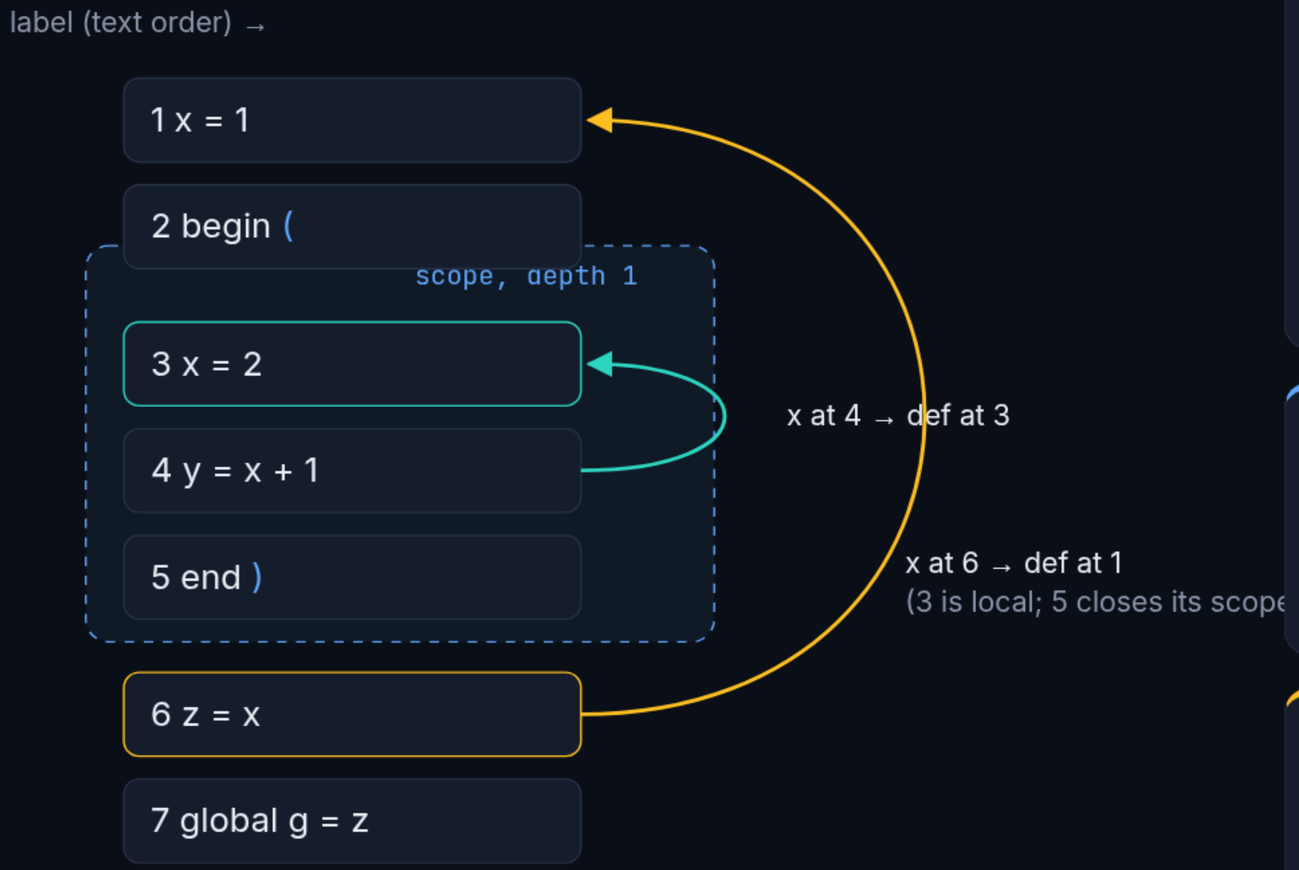
8
Workflow
scripts, lints, Reports, debugging

9
Numbers
100k lines, 1-16 threads

10
Open problems
where to look next

THE MODEL

Text order is for binding; edges are what run



A local reaches up to the close of its scope; a global reaches every later use.

Node

a pure application of a client `Operation` to argument text. Reads fields, defines fields, produces an output; may unfold children, claim followers, open or close scopes.

Use → reaching def

the nearest earlier **visible** definition, found in O(log n) from bracket summaries.

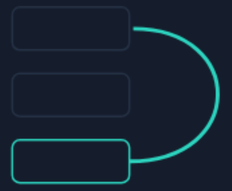
Wait

only for that definition's value, or for **pending structure** the binding walk crosses.

THE MODEL

The four rules

I



Order is used only for binding

Text order decides **which** definition a use sees, never **when** anything runs.

II



Nothing waits except on what its binding crosses

A pending node elsewhere in the program blocks nobody.

III



The schedule changes speed, never results

Ids are content hashes; canonical Report counters are schedule-independent.

IV



Evaluation is eager

Every node runs once its bindings are known. No demand counting (removed in `5e969d0`).

Consequences: **no current state**, **no stack**, **no phase that visits nodes in order**. A sequential interpreter has all three; that is what makes it the reference, not the model.

`git show 5e969d0 · src/lib.rs:9-22`

THE CLIENT CONTRACT

A client is a set of Operations

```rust
pub trait Operation: Send + Sync {
    fn name(&self) -> &'static str { "op" }
    fn defs(&self, args: &[ValueId], values: &Values)
        -> Option>;      // None = pending
    fn may_define(&self, ..) -> MayDefine { MayDefine::Any
    fn may_define_field(&self, .., f: Field) -> bool { true
    fn effect(&self, ..) -> Effect { Effect::None }
    fn claim_limit(&self, .., following) -> Option;
    fn claim(&self, .., following) -> Claim { Claim::None }
    fn until(&self, .., cand: OpId, cand_args) -> Until;
    fn until_value(..) -> bool;  fn until_values(..) -> boo
    fn until_predict(&self, .., following) -> Option;
    fn reads(&self, ..) -> Option> { None } // hint
    fn run(&self, cx: &mut Ctx<'_>) -> Result;
}
```

Hook	Called	Gives
`defs` `may_define` `effect` `claim` `reads` `until_predict`	once per distinct (op, text), memoized	structure known from text
`run`	every time inputs change	output, defs, children, dynamic structure

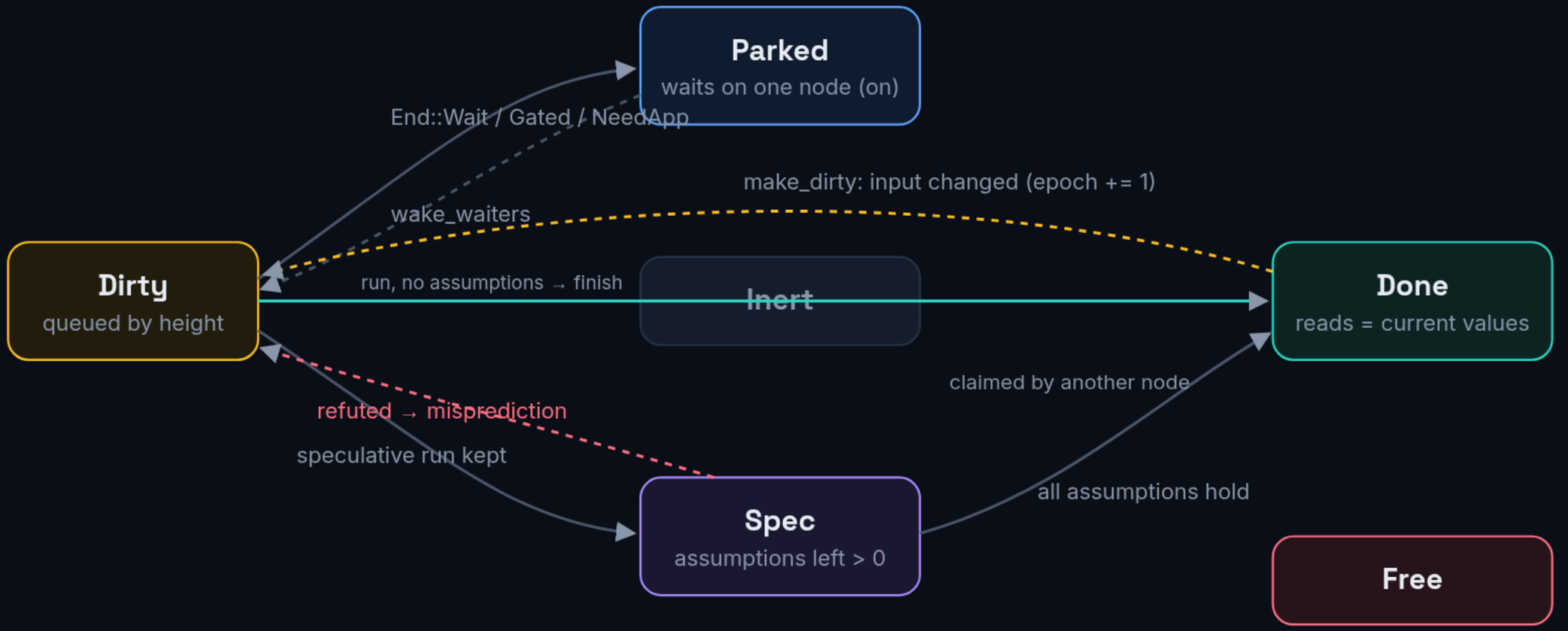
`Stop` is returned via `?` from any `Ctx` read whose value is not ready; the engine re-runs the operation **from the start** later. Operations must be pure.

Text-only hook answers are cached in `Engine::statics`; if something besides the text changes them, the client calls `Engine::refresh_where` (commit `e1ae33a`).

src/engine.rs:196-314 · src/ctx.rs:12-18

THE MODEL

A node's life: six states



in_work() = Dirty | Spec | Parked. Every change marks exactly the affected readers dirty, so at quiescence every Done node read current values.

src/engine.rs:455-463, 3489-3516, 4877-4990

EXAMPLE · BINDING

One small program, bound by text order

#	line	output
0	`x = 1`	`1`
1	`begin`	`()`
2	`x = 2`	`2`
3	`y = x + 1`	`3`
4	`end`	`()`
5	`z = x`	`1`
6	`global g = z * 10`	`10`
7	`w = g + 1`	`11`

`y = x + 1` (line 3) binds to `x = 2`: same scope, nearest earlier. Output 3.

`z = x` (line 5) is past the `end`, so line 2 no longer reaches it; it binds to `x = 1`. Output 1.

`global g` reaches every later use at any depth; `w` = 11.

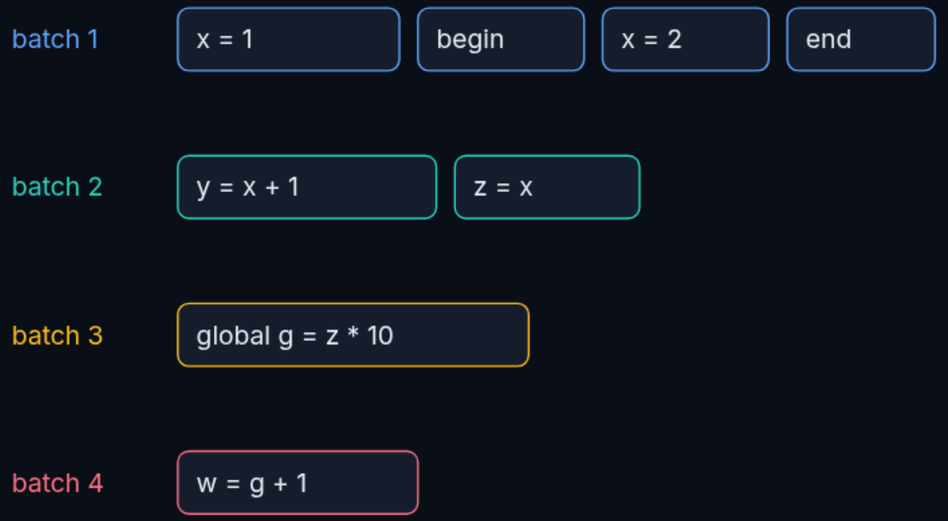
cold Report	
runs / nodes_run	8 / 8
steps (batches)	4
critical path	3
pending_waits, stops, gated	0, 0, 0

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs

EXAMPLE · COLD EVALUATION

The same program, batch by batch

critical path 3 → the longest chain z → g → w on top of x = 1



Each batch: run every node whose reaching definitions are settled (in parallel), then apply in NodeId order. Note batch 2 runs y and z together although they are 2 lines apart in the text: order only chose their bindings.

Batch membership drawn from the dependencies; the Report gives steps 4, critical path 3.

Rule I. The text says line 5 comes after line 3; nothing makes it run after.

Rule IV. Eager: nobody asked for `w`; it runs because its binding is known.

With `Threads(16)` the batches are identical; only the time per batch changes (rule III).

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs

EXAMPLE · INCREMENTAL EDITS

Four edits to that program, and what each one cost

edit (on the previous state)	nodes_run	cutoffs	rebinds	scope_region	why
`x = 2 → x = 5`	2	0	0	0	x(2) and its reader y re-run; z still binds to x = 1
`y = x + 1 → y = x + 0`	1	0	0	0	only the edited node; nobody reads y
`z = x → z = x + 0`	1	1	0	0	new text, same output 1: cutoff, g and w do not run
`end → q = 0 (scope never closes)`	4	0	1	4	z now sees x = 5 (rebinds 1); q, z, g, w run: z = 5, g = 50, w = 51 (scope_region 4)

Exactly the affected nodes. An edit inside the scope never touches readers outside it.

Early cutoff is per value, not per text: `z = x + 0` re-runs and stops there.

Bracket edit. Removing `end` moves bindings: `rescope_node` touches the 4 nodes of the region. All results checked against the interpreter.

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs

DATA

The Engine struct at a glance

Node store

`nodes: Vec<Node> ≤128 B each`
`aux: Vec<Aux> rare data`
`free_slots, free_aux`
`ids: NodeId → Slot`

Slot = (ix, gen); free bumps gen

Order

`order: Arena<OItem, OSum>`
`units[u].root (one treap / unit)`
`order_dirty (batched repair)`
`accs_arena: Arena<ValueId,AccSum>`

keyed by u64 label

Binding indexes

`local_ix (u, f, depth, label)`
`global_ix (u, f, label)`
`defs_ix (u, f, label)`
`users (site, f, label)`

BTreeMap: ordered range queries

Pending structure

`pending unbounded def sets`
`pending_f bounded to Names`
`pending_p bounded to Prefixes`
`pending_eff unknown scope effect`

Scheduling

`queue: Vec<Vec<(NodeId,Slot)>>`
`bucket queue by height`
`parked, claim_dirty, level`
`spec_live, spec_done, touch_log`

Values & memo

`values: Values (64 shards)`
`memo: (OpId, args) → Slot`
`text_memo, statics (RwLock)`
`environment, loop_vals, seqs`

Ownership rule

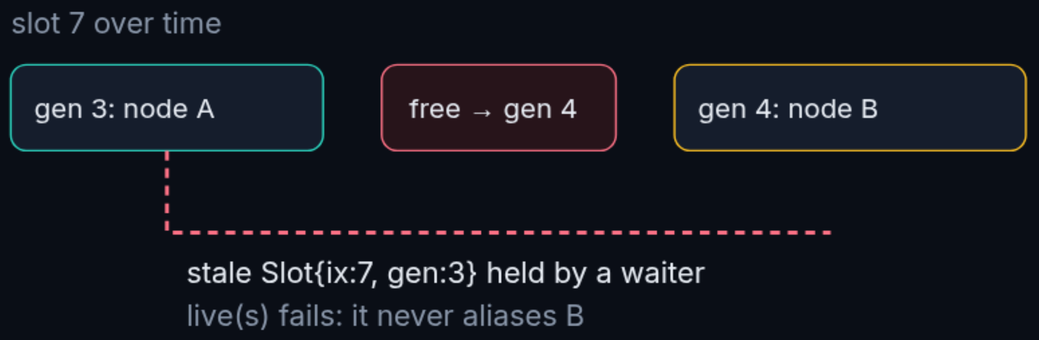
Jobs get `&Engine` (shared, read-only) in the parallel phase; only `apply` takes `&mut Engine`.
The RwLocks (memo tables, statics) are the only interior mutability.

src/engine.rs:833-935

DATA

Node record, Aux, and generation-checked slots

```rust
pub(crate) struct Node {
    pub(crate) id: NodeId,        // content hash
    pub(crate) args: ValueId,     // the text
    pub(crate) output: ValueId,
    pub(crate) label: u64,        // order key
    pub(crate) sig: u64,
    pub(crate) op: OpId,
    pub(crate) unit: u32,
    pub(crate) parent: Slot,
    pub(crate) height: u32,
    pub(crate) floor: u32,
    pub(crate) epoch: u32,        // bumped by make_dirty
    gen_: u32,                    // slot generation
    aux: u32,                     // NO_AUX or index
    on: Slot,                     // what it is parked on
    pub(crate) state: State,
    pub(crate) effect: u8,  bound: u8,  flags: u16,
    pub(crate) uses: Box<[Use]>,    // 16 B each
    pub(crate) defs: Box<[Def]>,
}
```



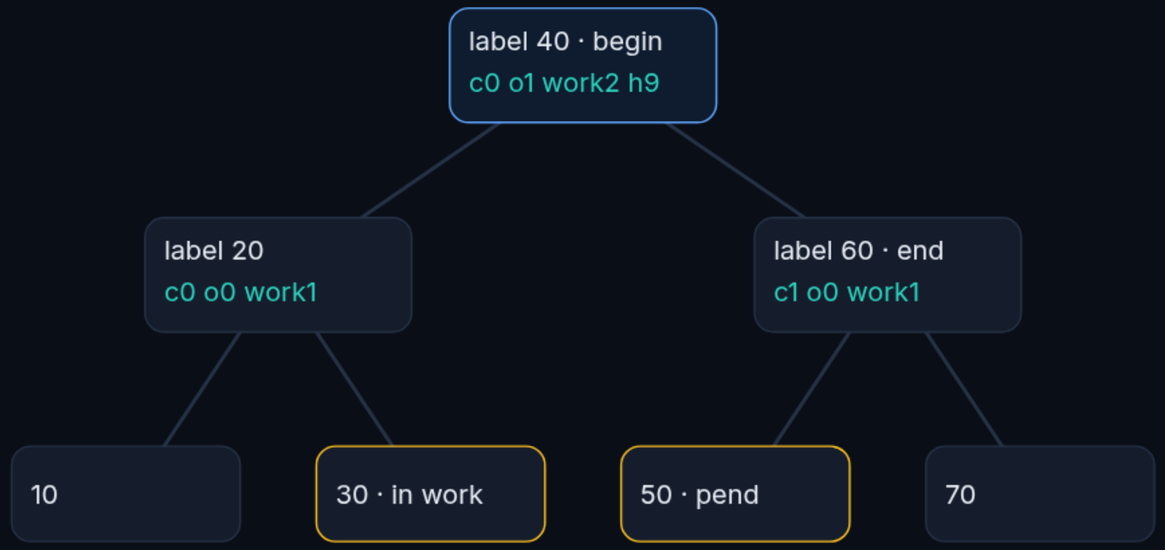
`Aux` (engine.rs:584-614): children, waiters, out_readers, claimed / claimed_by, assumers, pending index, stream range, `reach1` for pending claims. Only nodes that need it get one.

The generation check fixed a false `Cycle` when an eval replaced its child in a reused slot (commit `5e969d0`, test `slot_reused_steps_later_is_a_new_node`).

src/engine.rs:46-76, 524-551, 584-614

DATA · ORDER TREE

One augmented treap per unit, keyed by label



Priority = hash(label) → shape is a function of the keys.

Each node caches OSum of its subtree:

c/o unmatched closes/opens, work in-work count, maxh max height, pend pending count, maxr / maxhit furthest pending-claim reach.

Queries: fold(lo,hi) · search_back(pred) · search_fwd · set_many

all O(log n); search_back finds e.g. the nearest unmatched opener.

```
665  fn combine(&self, a: OSum, b: OSum) -> OSum {
666      OSum {
667          c: a.c + b.c.saturating_sub(a.o),
668          o: b.o + a.o.saturating_sub(b.c),
669          work: a.work + b.work,
670          maxh: a.maxh.max(b.maxh),
671          pend: a.pend + b.pend,
672          maxr: a.maxr.max(b.maxr),
673          maxhit: a.maxhit.max(b.maxhit),
674      }
675  }
```

Brackets compose like a monoid of `)…)(…(` shapes: a close in **b** first cancels an open of **a**. The depth at a label is just `fold(0, label).o`.

`OItem.hit` is `reach` or 0 while the claimer waits for a recompute: each pending claim is hit once per flush (`9b7fc36`).

src/tree.rs:1-60, 201-320 · src/engine.rs:616-691

Definitions, users and pending sets are ordered maps

Map	Key	Value	Asked by
`local_ix`	`(unit, field, depth, label)`	def slot	`walk`: one range query per enclosing scope level
`local_depth`	`(unit, field, label)`	depth	re-indexing when brackets move
`global_ix`	`(unit, field, label)`	def slot	`walk`: latest global before the use, at any depth
`defs_ix`	`(unit, field, label)`	def slot	all definitions of a field (re-binding on add / remove)
`users`	`(site, field, label)`	reader slot	who to dirty / move when a def changes (`move_user`)
`pending` · `pending_eff`	`(unit, label)`	slot	`walk`: unbounded def set / unknown effect in (site, use)
`pending_f`	`(unit, field, label)`	slot	def set bounded by `MayDefine::Names`
`pending_p`	`(unit, prefix hash, label)`	slot	`MayDefine::Prefixes`; `pending_p_lens` lists the lengths in use
`stream_ix`	`(unit, first label, slot)`	-	stream claimers whose scanned range an edit touches
`memo` · `text_memo`	`(OpId, args)` · `(text, text)`	slot · value	lexical applications, unfolded text

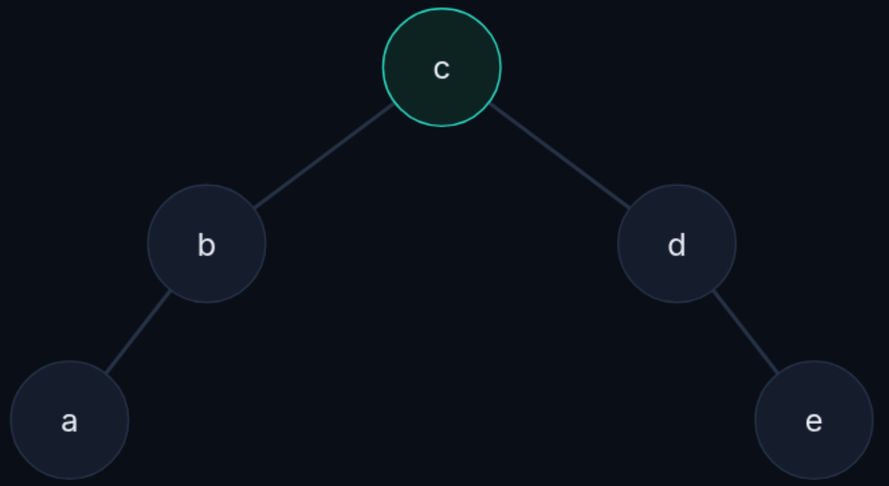
Commit de5d2a3: what remains in the sequential apply "is index mutation (order-tree inserts, definition and user indexes), not walks". These maps are the hot spot.

Values are content-hashed and interned

```rust
pub enum Value {
    Unit, Bool(bool), Int(i64), Str(Box),
    List(Box<[ValueId]>),
    Nil,            // the empty sequence
    Seq(SeqNode),   // canonical treap node
}
fn content_id(v: &Value) -> ValueId {
    // composites hash the ids of their parts
    Value::List(xs) => hash_words(4, &ids(xs)),
    Value::Seq(n) => hash_words(5,
        &[n.left.0, n.elem.0, n.right.0, n.prio]),
}
```

- Equal values ⇒ equal ids, in every process; a collision is detected and panics.
- `Values`: 64 RwLock shards + a per-thread direct-mapped read cache (4096 slots); a value read twice gets a private copy.
- `Field(ValueId)`: any value can name a field.

Seq [a b c d e]: a treap, priority = hash(elem)



Shape depends only on the elements, so
`seq_concat(ab, cde) == seq_concat(abc, de)`
same ValueId however it was built; concat shares both sides.
Accumulators use a polynomial hash summary (AccSum, mod 2^61-1).

src/value.rs:1-120, 437-474 · src/engine.rs:693-786

Two hashers, both deterministic

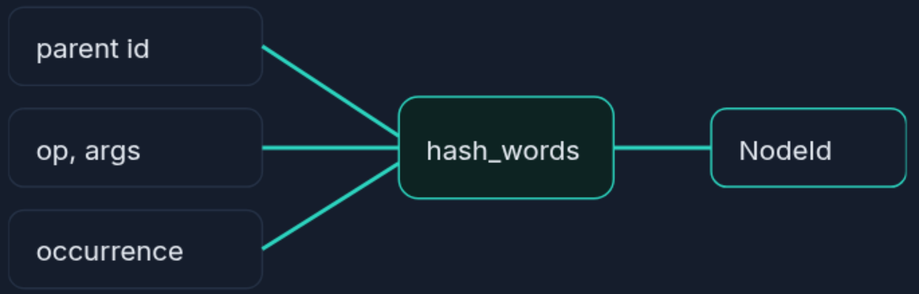
FxHasher

For internal tables (`FxHashMap`, `FxHashSet`). Keys are mostly hashes already. `rotate_left(5) ^ w, * K`. No random seed, so iteration order is reproducible run to run.

```
fn add(&mut self, w: u64) {
    self.0 = (self.0.rotate_left(5) ^ w)
        .wrapping_mul(FX_K);
}
```

hash_words / hash_bytes

Strong mixer (wyhash-style 64×64→128 "mum") for content ids: `ValueId`, `NodeId`, child ids, the environment.



Equal environments give identical ids and results (tested).

THE LOOP

evaluate → rounds → drain

evaluate

restart_loops() if feedback
apply_environment()
touch_log.clear()
evaluate_inner → rounds
feedback appeared? cold path
collect_apps(false)
sweep_values()
finalize_report()

Ok(Report) or EvalError:
Cycle(ids)
NonConvergence{rounds}
ConflictingPublication{field}

rounds (loop-back)

drain()
parked left? collect_apps(true)
else Err(Cycle(parked))
for each loop field:
publishers agree?
changed vs loop_vals?
none changed → Ok
dirty_loop_users(f), again

One round per loop-back iteration;
capped by set_max_rounds.

drain (one step / iter)

flush_claims()
settle_deferred(); flush_order()
queue_first() → (level, nodes)
revalidate F_REBIND nodes
collect spec candidates
sched.select → batch
sanitize(batch)
exec.execute(run_job) ‖
results.sort_by_key(node)
for r: apply(r) sequential
settle_deferred()

Returns when the queue is empty.

src/engine.rs evaluate (≈5157), rounds (≈5294), drain (≈5408)

THE LOOP

The batch cycle



Level: the lowest non-empty bucket of the height queue. A `Done` node lower than every queued node is **settled**; non-speculative runs only read settled values.

Run: `run_job(&self, job, fuel)` builds a `Ctx`, does the gate walks, calls `op.run`. Every read becomes a recorded `Use`.

Apply: results sorted by `NodeId` before commit, so the commit order does not depend on which thread finished first.

"At least one ready node always runs" (`sanitize`): a scheduler cannot stall progress.

src/engine.rs drain, sanitize, run_job

THE LOOP · CODE

The heart of drain

```
loop {
    self.flush_claims();
    self.settle_deferred();
    self.flush_order();
    let Some((level, at_level)) = self.queue_first() else { return
    self.level = level;
    for s in at_level {
        if dirty(s) && has(F_REBIND) && self.revalidate(s) { contin
        if dirty(s) { ready.push(s); }
    }
    // speculative candidates: higher levels + parked nodes
    sched.select(&just_ids, &cands, &mut batch);
    Self::sanitize(&ready_ids, &cands, &mut batch);
    {
        let this: &Engine = self;
        exec.execute(batch.len(), &|i| this.run_job(batch[i], fuel)
    }
    results.sort_by_key(|r| r.node);
    for r in results.drain(..) { self.apply(r); }
    self.settle_deferred();
}
```

src/engine.rs drain (condensed; profiling spans and counters removed)

Claims first. Structure edits queue claimers in `claim_dirty`; recomputed once per flush.

One level. Nodes dirtied only by re-binding (`F_REBIND`) are revalidated, which is cheaper than any run.

Policy. `Scheduler` chooses; `sanitize` dedups and forces one ready job.

Parallel. A shared `&Engine`; the `Executor` decides threads.

Sequential. Deterministic commit order. ~1 s of 1.34 s cold at 16 t.

Heights order work; the queue is a bucket array

queue: Vec<Vec<(NodeId, Slot)>> (entries removed lazily)



Who sets a height

- End::Raise(h) / GateRaise(h): a read target is not settled below h
- End::Wait(t), t done: height = t.height + 1
- finish(x, target_height): above what it read; raise() lifts readers
- child_heights: unfolded children start above hinted inputs

grey entry = stale (node no longer F_QUEUED at that height)

Invariant

A `Done` reader is higher than what it read. So the level is a frontier: everything below it is final.

Critical path

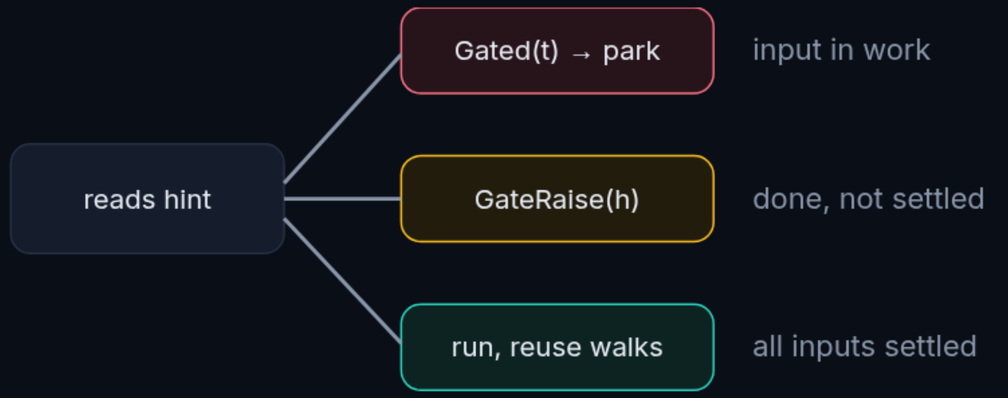
`Engine::critical_path()`: the dependency depth the heights reached. 17 for the 10k typing program, 44 at 100k (runs below). Steps ≈ number of levels visited: 99 at 10k, 422 at 100k.

Heights are estimates

"It decides when a node is tried, never a result" (child_heights doc).

The gate: read hints checked before a node starts

```rust
fn gate_walks(&self, x: Slot)
    -> Result, End> {
    let n = self.n(x);
    if !n.has(F_HINT) || n.has(F_APP) { return Ok(vec![]); }
    let Some(reads) = self.statics(n.op, n.args).reads.as_re
        else { return Ok(vec![]) };
    for &f in reads {
        if self.loop_vals.contains_key(&f) { continue; }
        let w = self.walk(n.unit, n.label, f);
        if w.pending != NIL {
            return Err(End::Gated(w.pending));
        }
        if w.site != NIL {
            let t = self.n(w.site);
            if t.in_work() { return Err(End::Gated(w.site));
            if t.height >= self.level {
                return Err(End::GateRaise(t.height + 1));
            }
        }
        walks.push((f, w));
    }
    Ok(walks)
}
```



The gate runs **inside** the parallel job (read-only). Its walks are handed to the run, so `use_field` does not walk the same field again: gate walks were 23% of cold (de5d2a3, 3.16 s → 2.72 s at 1 thread).

Report: `gated` counts nodes held back. In the 10k run below: 4057 gated of 11256 runs.

src/engine.rs gate_walks · src/ctx.rs use_field

Scheduler picks, Executor runs

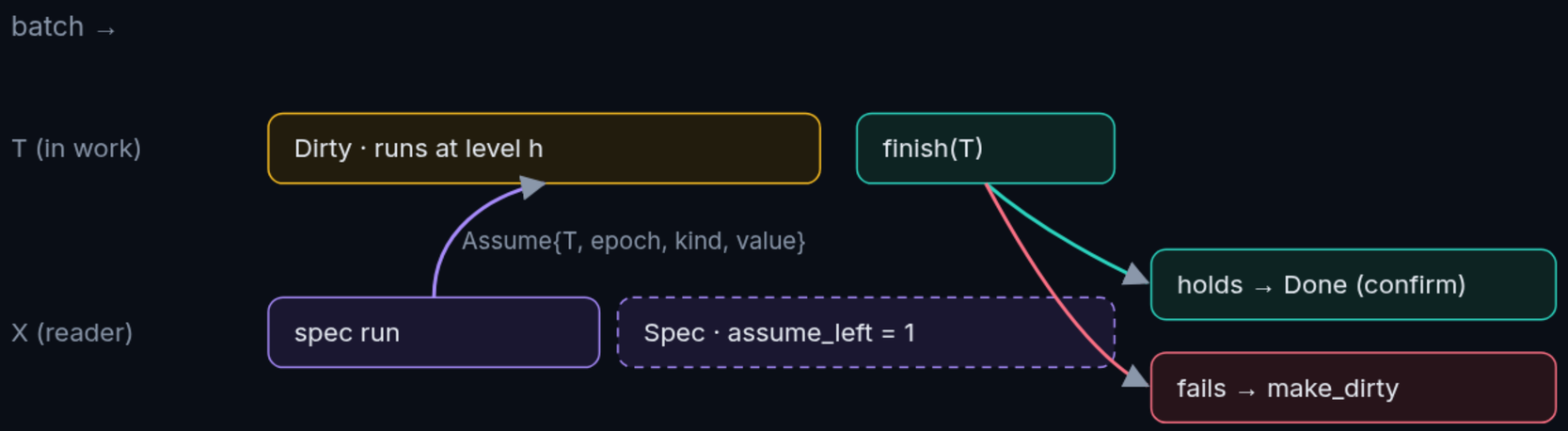
```rust
pub trait Scheduler {
    fn speculation_budget(&self) -> usize { 0 }
    fn max_outstanding(&self) -> usize { 4096 }
    fn fuel(&self) -> u64 { 10_000 }
    fn select(&mut self, ready: &[NodeId],
              candidates: &[Candidate], batch: &mut Vec);
}
pub trait Executor {
    fn execute(&mut self, n: usize,
        run: &(dyn Fn(usize) -> RunResult + Sync),
        out: &mut Vec);
}
pub enum Speculation { Off, Predicted, All }
pub struct Candidate {
    pub node: NodeId,
    pub has_prediction: bool, // ran before / F_PREDICT
    pub waiting: bool,        // parked
}
```

Impl	Where	Behaviour
`Greedy`	sched.rs	all ready; speculate per `Speculation`, `budget`/step
`RandomScheduler`	sched.rs	random subsets, random fuel (3-66): tests schedule-independence
`Sequential`	sched.rs	executor: jobs in order on the calling thread
`Threads(n)`	examples/common	`std::thread::scope`, dynamic grains of 4-64 jobs; serial under 64 jobs

The library has **no threads** (wasm32). Parallelism is the client's `Executor`. `Engine::evaluate_sequential()` = `Greedy::default()` + `Sequential`.

src/sched.rs:1-226 · src/lib.rs:52-61 · examples/common/mod.rs Threads

Speculation: run early, record assumptions, confirm in bulk



AssumeKind

`Def(f)` value of T's def f

`Output` T's output

`Structure(f)`
pending structure resolved without re-binding the use; may re-attach to next pending

Predictions are the previous run's values and shape (has_prediction) or a stream claim's until_predict. Shape changes (children, claim, effect, defs) are provisional (F_PROV
Out of fuel → misprediction + F_NOSPEC. Assumption on a freed or claimed node → refuted (refute_assumers). A target finishing earlier in the same batch → stale_assumpt
Report: speculations · confirmations · mispredictions · fuel_outs · spec_shape_changes.

THE LOOP · APPLY

apply(RunResult): the commit, in order

stale? (epoch, freed read site, stream touched) → make_dirty

speculative? fuel out / assumptions already false → drop

End ≠ Output: Gated · Raise · Wait · NeedApp · NeedChild · WaitRange

set_children (unfold)

set_uses (users index)

set_stream

set_defs → changed?

change_effect (dyn)

claim_count (dyn claim)

output ≠ old → dirty readers

publishes · contribs · watch

re-dirtied while applying? enqueue again

assumptions left → Spec

finish(x, target_height)

finish: Done · sync_pending · effect region · height/raise · wake_waiters · settle range readers · confirm/refute every assumption on the node

src/engine.rs apply (≈4529-4797), finish (≈4886-4999)

Early cutoff

Readers are dirtied only if the output or a definition's value actually changed. `cutoffs` counts runs whose output and definitions came out unchanged.

Two-phase apply

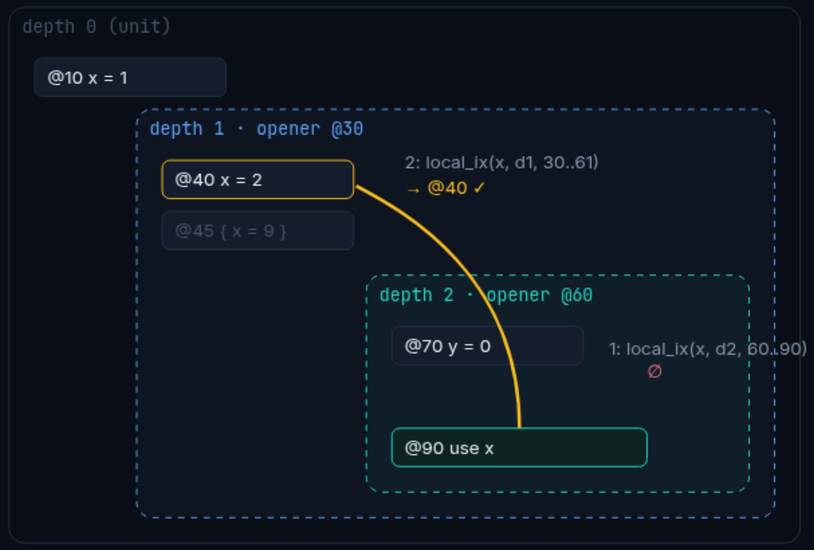
Child start heights are computed in the parallel phase (`child_heights`); the commit adds in-text siblings and binds without walking (`bind_fresh`) (de5d2a3).

Waking

A parked node sits on exactly one node (`on`). `release_waiters` is shared by `free_node` and `make_inert` (418470d fixed a missing wake).

Depth-indexed lookup: one range query per enclosing scope

use of x at label 90, depth 2

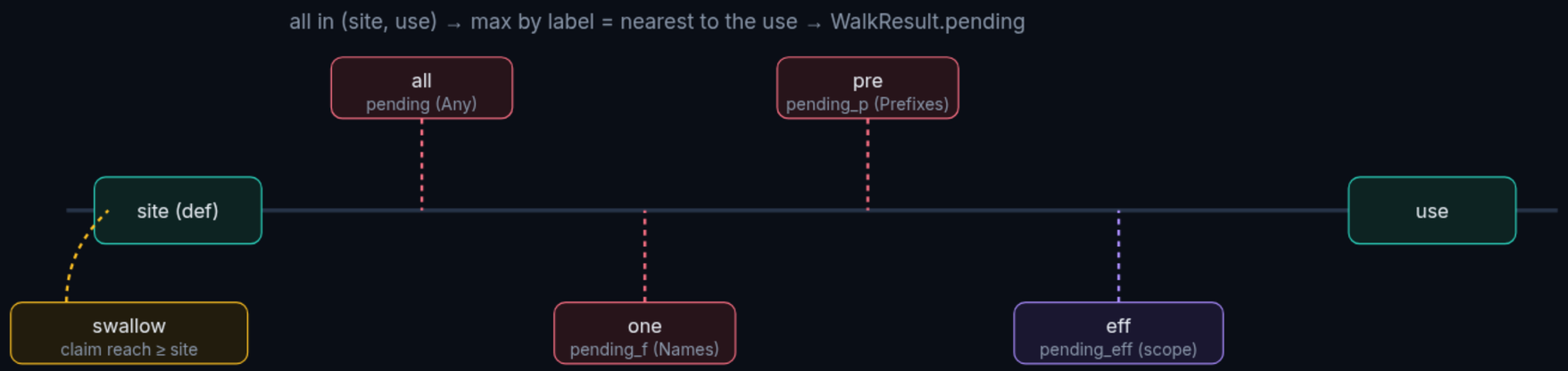


@45 sits in a closed sibling scope at depth 2: never visited. Globals: one extra global_ix range;

```rust
let mut depth = self.order.fold(&OAug, root, 0, bhi).o;
let mut hi = label;
loop {
    let opener = self.order.search_back(&OAug, root, bhi, &|s| s.o >= 1);
    let lo = opener.unwrap_or(0);
    if let Some(..) = self.local_ix
        .range((unit, f, depth, lo)..(unit, f, depth, hi)).next_back()
    { local = Some(..); break; }
    match opener {
        Some(k) if depth > 0 => {
            bhi = k; hi = (k + 1).min(hi); depth -= 1;
        }
        _ => break,
    }
}
```

"A local definition reaches the use iff it lies directly in one of the scopes enclosing the use: one range query per enclosing level, never a scan over closed scopes." Unmatched closes at depth 0 are ignored.

Five ways a binding can cross something undecided




```
let pending = [all, eff, one, pre, swallow
    .into_iter().flatten()
    .max_by_key(|x| x.0)
    .map_or(NIL, |x| x.1);
WalkResult { site, pending }
```

Non-speculative `use_field` with `pending ≠ NIL` → `End::Wait(pending)`: the node parks on that one node. Rule II in code.

`walk_effects` is O(log n) per use (0c7cd92): only effects whose scope could hide the site count; load had gone 2.0 s → 10 s before that fix.

src/engine.rs walk (tail), walk_effects, walk_prefixes, pending_for · src/ctx.rs use_field

Bounding pending structure from text

MayDefine	Uses that wait on the pending node
`Nothing`	none (nothing visible after it)
`Names(fs)`	only uses of those fields (`pending_f`)
`Prefixes(ps)`	only fields starting with a prefix; toy `eval "e1_ = " + x`
`Any`	all, filtered by `may_define_field`

Advisory: if a node defines outside its bound anyway, the uses it reaches are re-bound and re-run when it finishes. "Speed, never results."

Claim	meaning
`None` / `Count(n)`	from text: the next n siblings go Inert
`Pending`	decided by `Ctx::claim(n)`; bounded by `claim_limit` → `reach1`

```rust
pub enum Effect { None, Open, Close, Pending, Scope(Brackets) }
pub struct Brackets {
    pub closes: u8,          // at most 3
    pub opens: u8,           // at most 3
    pub before: bool,        // node lies inside what it opens
    pub kind: Option,
}
// packed: closes bits 0-1, opens bits 2-3, before bit 4
```

after (begin): `begin` (`a = 1`

before (item): (`item a = 1` `item b = a`

A node opening before itself defines inside its own scope.

src/engine.rs:96-195 (Locality, Effect, Brackets, Claim, Until, MayDefine)

eval: structure known only when it runs

#	line	output
0	`a = 1`	1
1	`eval "b = " + a`	()
2	`c = b + 1`	2
3	`d = a + 2`	3

run	nodes_run	rebinds	created / freed	pending_waits
cold	5	0	-	0
`a = 7`	5	1	1 / 1	-

The eval's text is a value, so what it defines is pending until it runs. Its literal prefix `"b = "` bounds it: `MayDefine::Prefixes(["b"])` (5e969d0).

`d = a + 2` never waits on it (field `a` is outside the bound); `c = b + 1` does. 5 runs = 4 lines + the unfolded `b = 1` child.

Edit `a = 7`: the eval's text changes, its child `b = 1` is freed and `b = 7` created (new content id), `c` re-binds to it: rebinds 1, c = 8.

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs

Scope kinds and implicit scopes



How kind() works

The opener defines a hidden local field (kind_field) inside its scope. `Ctx::scope_kind()` is just `use_field(kind_field)`: a binding to the innermost opener, re-bound like any other use. A pending effect becomes structure when its node is done. Fresh balanced text gets kinds without reader capture: 229 ms → 13 ms at 100k (dc76fd8).

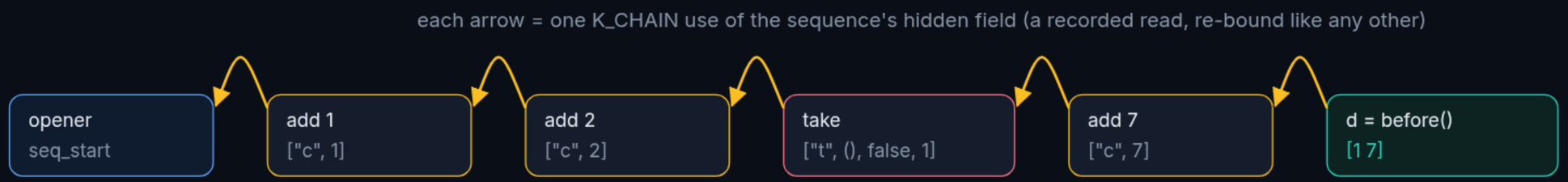
Test (tests/engine.rs)
`implicit_scope_opened_by_a_starter_and_ended_by_a_closer`
`closing_the_enclosing_scope_closes_the_implicit_one`
`kind_read_binds_across_an_unfold`
`a_pending_close_waits_readers_of_an_openers_marker`

Trace format keys: `scope_kind`, `starts_implicit`, `closes_implicit`, `scope: {closes, opens, before}`.

Fix on the way: "a pending close could hide a use's own site without the use waiting".

src/ctx.rs scope_kind · src/engine.rs set_kind/sync_kind · tests/engine.rs:187-245

Contributions: a sequence as a chain of bindings



A declared sequence (Engine::declare_sequence) is a hidden local field. Contributor defines it statically; the opener defines a start marker inside its scope.

Edit "add 2" → only readers whose chain passes through it re-run (test a_middle_contribution_edit_reruns_only_its_readers).

Ctx call	Toy	What it reads / defines
`push_contribution(seq, e)`	`add e`	defines hidden field = `["c", e]`
`prev_contribution(seq)`	`prev()`	one binding: the contribution right before (None at the scope start)
`contributions_before_me(seq)`	`before()`	walks the chain back to the start marker; a link crossing pending structure waits
`take_prev_contribution(seq, r)`	`take` / `replace`	defines `["t", r, has_r, visible]`: later readers bind to the taker

A contribution edit re-runs only its chain readers

#	line	output
0	`add 1`	`1`
1	`add 2`	`2`
2	`a = before()`	`[1 2]`
3	`take`	`2`
4	`b = before()`	`[1]`
5	`add 7`	`7`
6	`d = before()`	`[1 7]`

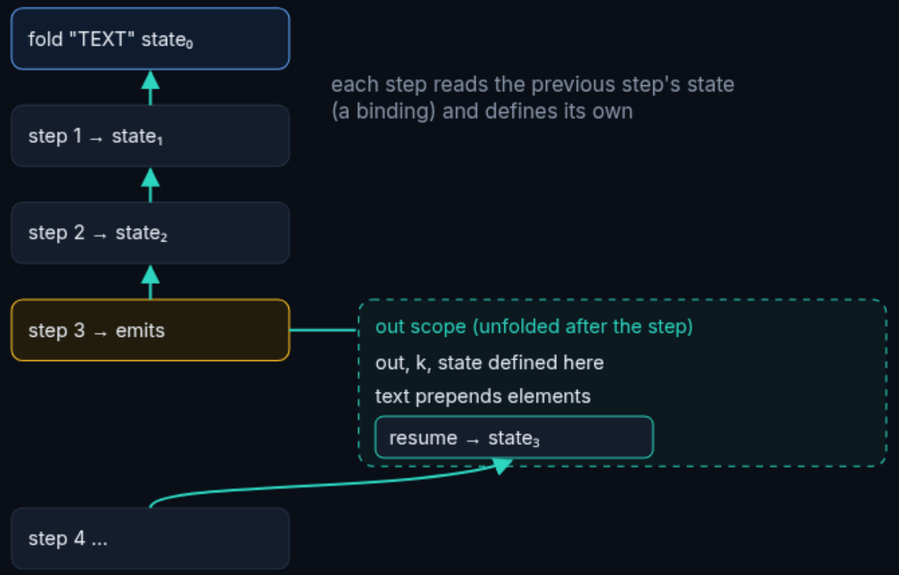
run	nodes_run	cutoffs	pending_waits	stops
cold	7	0	3	4
`add 2 → add 20`	5	2	-	-

After the edit: `a = [1 20]`, `take` takes 20, but `b` and `d` come out unchanged (`[1]`, `[1 7]`): the 2 cutoffs.

Cold: readers of `before()` start, stop on an unrun link, park (4 stops, 3 pending waits), and are woken; they could not bind past an unrun contributor because sequences are declared (95dc2e6).

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs

Resumable fold, from ordinary bindings



No global re-evaluation, no loop-back rounds: one output at a time.

Why it is incremental

An edit re-runs steps until the state is **identical**, then early cutoff stops it (test `a_fold_edit_recomputes_only_until_the_state_realigns`).

Toy surface

`fold "TEXT"` and `when C STMT`; the interpreter mirrors it. Test `fold_over_a_thousand_elements_emits_48_outputs`.

Trace surface

```
"fold": {"sequence": "items", "step_class": "add"}
```

Claims: a node takes the text that follows it

inert claim (listuntil, sumnext, listto)



taken nodes go Inert: they do not bind, run or define

stream claim (collect I until mark)



taken nodes still run; claimer reads their outputs after unfolding:

I = [1 2 3] (tests/engine.rs:358)

Until	candidate test
`No` / `Yes`	from the candidate's class (op + text)
`Read(f)`	`until_value` on f's value at the candidate
`ReadEach(fs)`	`until_values`; each is a recorded read

Kind	Extent known	Toy
Known / `Count(n)`	from text	`sumnext s 2`, `listto`
`Pending`	when the claimer runs (`Ctx::claim`)	`listuntil l v`
Stream	scan via `Ctx::claim_stream`	`collect … until C`
Predicted	`until_predict` → speculate	`… guess N`

A pending claim makes waiters only of uses whose reaching definition it could swallow: `claim_limit` → `Aux.reach1` → `OItem.reach` → `OSum.maxr` → the **swallow** source in `walk`.

Prediction confirmed by the recorded reads: a mismatch on any `confirm` field is a misprediction (7229195). `STREAM_RUN_LIMIT` = 4096 runs per evaluation counts as a cycle.

EXAMPLE · CLAIMS

An inert claim and a stream claim

#	line	output
0	`sumnext s 2`	`7`
1	`3`	`-`
2	`4`	`-`
3	`t = s * 2`	`14`

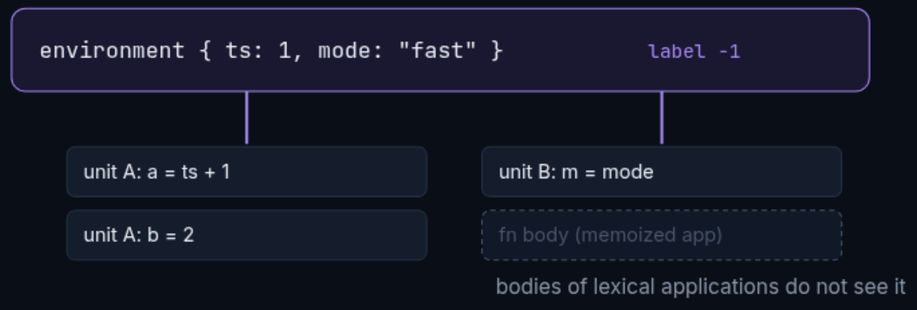
Inert. Lines 1-2 are claimed by count, from the text: they never run (output `-`). Edit line 1 to `30`: nodes_run 1 (the claimer, s = 34) and t follows: 68.

#	line	output
0	`collect l until mark`	`[1 2 3]`
1	`1`	`1`
2	`eval "2 3"`	`()`
3	`mark a`	`()`
4	`4`	`4`
5	`mark b`	`()`
6	`z = l`	`[1 2 3]`

Stream. Taken nodes still run; the claimer reads outputs **after unfolding** (the eval's 2 and 3) up to the first `mark`. Edit line 3 to `5`: the claim runs on to `mark b`: `[1 2 3 5 4]`, nodes_run 3.

measured: a scratch driver over examples/common (Program::load, set_line, evaluate_sequential), checked against interp.rs · same programs as tests/engine.rs claim / stream tests

The environment lives at text order −1



Engine::set_environment(iter) is staged (env_next) and applied at the start of the next evaluate: diffed against the previous one; only readers of changed fields re-run. A use with no site reads start_value (env, then set_env).

Interpreter mirror: interpret_env(lines, env). Trace: "meta.env" + "env_reads".

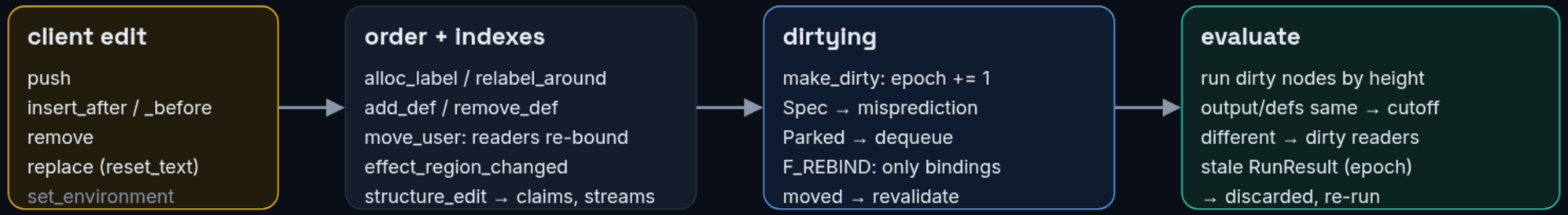
```rust
// examples/common/mod.rs
pub fn env_values(eng: &Engine, env: &[(&str, i64)])
    -> Vec<(Field, ValueId)>;
eng.set_environment(env_values(&eng, &[("ts", 2)]));
eng.evaluate_sequential()?;
```

Ids are content-derived, so equal environments give identical ids and results.

Tests:
`equal_environments_give_identical_ids_and_results`,
`changing_one_environment_field_reruns_only_its_readers`.

INCREMENTALITY

From an edit to the next quiescent state



Epochs

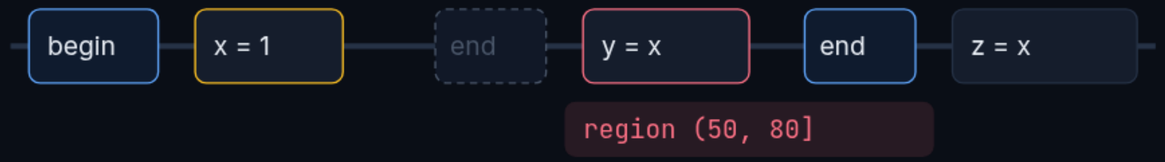
Every RunResult carries the node's epoch at start. apply() drops results whose epoch moved (an input changed while it ran). An Assume carries the reader's epoch too.

Revalidation A node dirtied only because a binding moved checks its recorded reads against the new sites first: often no run at all.

typing edit kind (10k, this run)	median	nodes run	rebinds	what dominates
insert char	456 µs	4	1	the edited line and its readers
change far def	7.7 ms	908	1	a widely read global: every reader really changes
edit in claim	255 µs	0	0	claimed text, claimer cut off

Bracket edits re-bind only what moved

moving a close from @50 to @80



Before (3505376 … f731bb8)

remove + re-add every definition in the region; each def's readers (much of the program for a top-level name) moved away and back, dirtied.

Now: rescope_node

- a local is re-scoped in place: readers leave only past its new scope end, arrive only from the definition that reaches it
- globals untouched (their reach ignores brackets)
- a contribution-chain reader re-runs only if one of its links moved

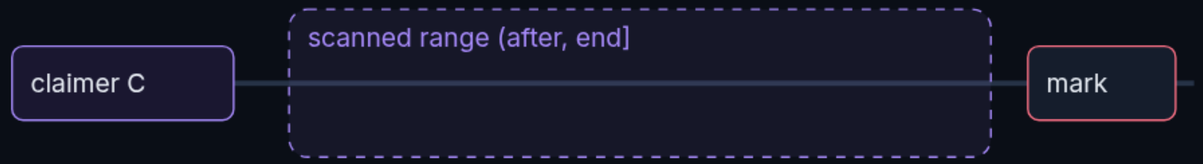
free_span: a balanced run of unfolded children is cleared then freed without region work (9b7fc36



typing 100k, 30 rounds (897e7ca). "The slow edits now execute 2-10 nodes (were 6k-17k), and what remains is proportional to the region."

Report: `scope_region` = nodes re-bound because a boundary moved; probe `RegionNode`.

Stream claims and the touch log



edit at label L in range

stream_touch(unit, L):

1. touch_log.push((unit, L)) during eval
2. stream_ix range → claimers to dirty

apply(r) of a claimer that ran meanwhile:

touch_log[r.touch_seq..] hits (after, end+1]?

→ StaleSite, make_dirty, run again

relabels re-key stream_ix (restream)

touch_log cleared at evaluate start/end

Why a log

A stream claimer's run reads a range of **structure**, not just fields. Other jobs in the same evaluation can unfold or free nodes in that range after the claimer started; the log lets `apply` detect that without locking.

Cycle guard

A taken node that reads the claim back never parks: each run feeds the next. `STREAM_RUN_LIMIT` (4096 runs) per evaluation reports a `Cycle`.

One statement per line, one feature per keyword

```
x = 1 + y                  assignment (local)
global g = 2               global assignment
p = {x: 1, w: 10, c: 3}    record: fields p.x, p.w, p.c
r = layout(p)              reads only p.x and p.w
{ a = 1  b = a }           block (unfolds into begin .. end)
begin / end                scope opened / closed apart
if c { .. } else { .. }    only the selected branch exists
f = fn(a) { .. }           lexical function value
y = f(2)                   memoized application
proc p(a) { .. } / do p(5)   unfold body at call site
eval "z = 4"               runtime text, unfolded in place
sumnext s 2                claims the next 2 lines
listto l .. endlist        claims up to endlist
listuntil l v .. mark k    pending claim
meaning put = "set"; ~put x 5   meaning is a field
emit e / page              accumulator `out`
loop later = 0             loop-back field
```

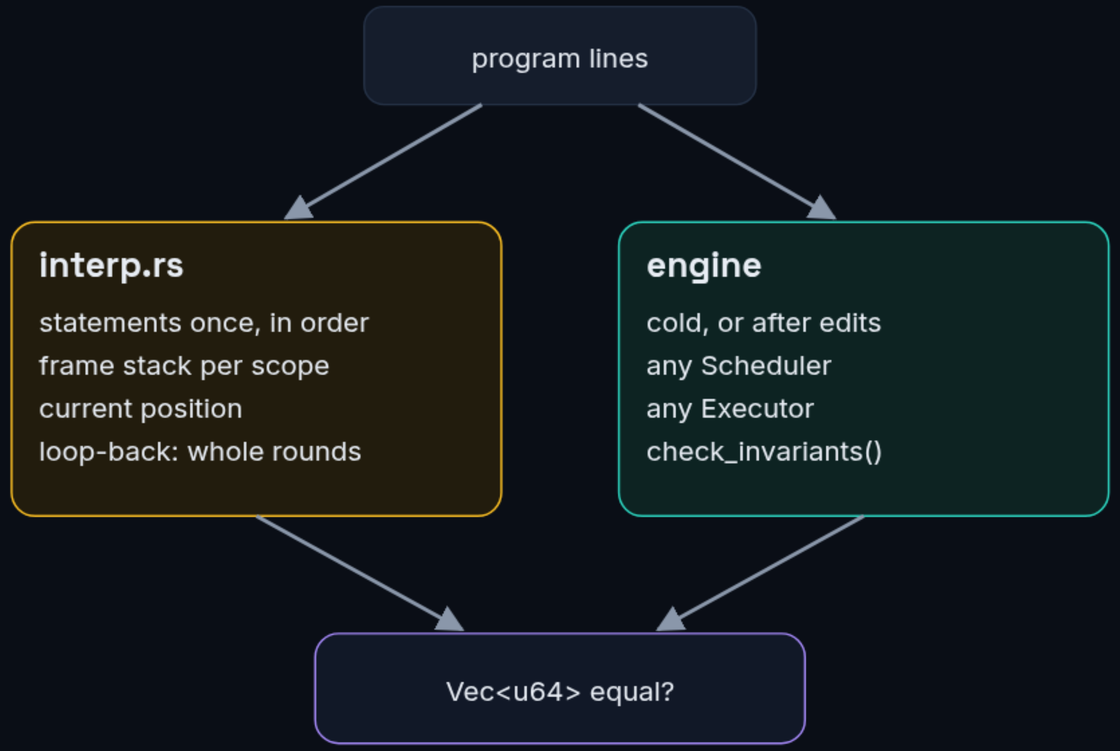
Added later	Feature exercised
`scope K`, `kind()`, `item`, `para`	scope kinds, implicit scopes
`add`, `take`, `replace`, `prev()`, `before()`	contributions
`fold "TEXT"`, `when C STMT`	resumable fold
`collect N until C [if F [and F]] [guess N]`	stream claims, prediction

`Lang`, `Program::load(lines)`, `set_line`, `insert_line`, `result(&eng)`; `generate(seed, n)` writes 24-line sections using every feature (0.3% computed `eval` lines).

`do NAME` is bounded by a program-wide procedure table; changing it calls `refresh_where` (e8c1951).

examples/common/mod.rs:1-26 (grammar), Program (≈1792), generate (≈2015)

The interpreter has everything the engine refuses to have



tests/engine.rs group (72 fns)	examples
binding & scopes	`reaching_definitions_across_scopes_and_globals`, `unmatched_close_is_ignored`, `moving_a_close`
incrementality	`early_cutoff`, `rebinding_on_insert_and_remove`
pending sets	`pending_definition_sets`, `bounded_do_and_eval`
claims	`claim_with_pending_delimiter`, stream / predicted tests
schedules	`schedule_independence_ids_and_speculation`, `wrong_shape_prediction_rolls_back`
whole programs	`generated_programs_match_the_interpreter`
fixpoints	`fixpoint_convergence_and_errors`, `cycles_are_reported_and_recoverable`
traces	`the_trace_loader_rejects_malformed_input`, round-trip, hand-written trace

Each regression found by fuzzing or by the client becomes a named test (commit bodies list them).

"It shares only the parser and the pure value helpers with the engine's operations; binding, scopes, claims, unfolding, accumulators and loop-back rounds are its own." (interp.rs:1-4)

examples/common/interp.rs · tests/engine.rs

Random edits until something differs, then minimise



typing --check failure → target/fail/before.txt + edits.txt (Program.edit_log) · chains failure → target/fail/lines.txt

SCHED=seq|spec|random selects the fuzz scheduler; examples/shrink.rs tries seq/spec/random until one reproduces.

```
# the usual repro after a failed typing check
scripts/nc.sh build --release --example shrink
scripts/run.sh ./target/release/examples/shrink \
    target/fail/before.txt target/fail/edits.txt
#  [--sched seq|spec|spec:T|random] [--out DIR]
```

191d61f: a `page`-reader anchored at a claimed sibling, found by checked edits at 100k lines on the cluster, "shrunk to 24 lines with the new tool; the case is a test."

typing, chains, and JSONL traces

typing.rs

1k / 10k / 100k / 1M-line generated programs: cold, then single-edit replays (insert / delete char, change far def, insert def, redefine meaning, edit in claim, move block close), each checked bytewise against cold unless `--no-check`. `--scaling`, `--bounded`, `--trace FILE`.

chains.rs

N pending claims in a row, each one's extent known only from the previous claim's result. `inert` (listuntil) and `stream` (collect) shapes; three edits checked against the interpreter. `SPEC=off|predicted|all`.

```
{"meta": {"version": 1, "document": "d",
  "steps": 2, "sequences": ["items"]}}
{"id": "s1", "class": "macro", "pos": 1,
 "reads": ["a"],
 "writes": [{"field": "x", "locality": "local"}],
 "scope": "open", "scope_kind": "A"}
{"id": "s2", "class": "use", "pos": 2,
 "reads": ["x"], "writes": [],
 "claim": {"until": {"class": "mark",
           "inert": true, "confirm": ["go"]}}}
```

replay.rs: strict loader (unknown keys, types, duplicate ids, non-increasing pos, bad depths rejected with the line). Edits: `change_writes`, `change_reads`, `insert_after`, `delete`. `--from-toy N` derives a trace from the toy.

examples/typing.rs:1-12 · examples/chains.rs:1-17 · examples/common/trace.rs:1-50 (format v1) · examples/replay.rs:1-13

`--features profile`: counters, spans, Chrome traces

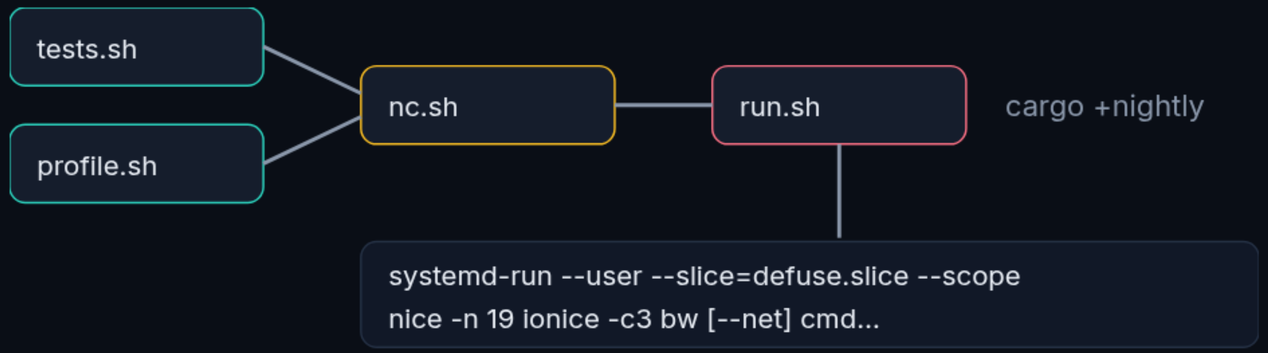
```
# release code + full debug info + probes
scripts/profile.sh 10000 --edits 5 --no-check
#  = nc.sh build --profile profiling --features profile \
#      --example typing
#    run.sh target/profiling/examples/typing \
#      --trace target/trace.json ...
# open target/trace.json in ui.perfetto.dev
```

- Without the feature every probe expands to nothing: release pays zero (test `probes_count_only_in_profile_builds`).
- Counts and span totals are per thread, merged at thread exit or `snapshot`: no cross-thread contention in what they measure.
- On wasm32 spans are counted but not timed (no clock).

Counters (obs::Counter)	Spans (obs::Span)
`Walk`, `WalkLevel`	`Evaluate`
`Step`, `Job`, `SpecJob`	`RunBatch`, `ApplyBatch`, `Job`
`Gated`, `Park`, `Wake`, `Dirty`	`Gate`, `Select`
`MoveUser`, `RegionNode`, `StaleSite`	`Region`, `Uses`, `Deps`
`RevalidateOk`, `RevalidateMiss`	`Children`, `Defs`, `Finish`
`ClaimRecompute`, `Create`, `Free`	`FlushClaims`, `FlushFrees`, `Claim`
`ValueGet`, `ValueGetMiss`, `ValueIntern`, `Sweep`	`Cycles`, `Sweep`, `Edit`

In code: `count!(Walk)`, `span!(ApplyBatch)` (macros from `crate::obs`).

Never call cargo or a binary directly



Task	Command
build	`scripts/nc.sh build --release --examples`
tests, each alone with a timeout	`scripts/tests.sh [regex]` (`PROFILE=debug`, `TMO=60`)
a benchmark	`scripts/run.sh ./target/release/examples/typing 10000 --edits 2 --no-check`
fmt	`scripts/nc.sh fmt` (rustfmt.toml: max_width = 120)
clippy, default	`scripts/nc.sh clippy --all-targets -- -W clippy::pedantic`
clippy, profile	`… clippy --all-targets --features profile -- -W clippy::pedantic`
clippy, wasm32	`… clippy --lib --target wasm32-unknown-unknown -- -W clippy::pedantic`
fetch crates (none today)	`scripts/nc.sh --net …`

The slice

`defuse.slice`: `MemoryMax=32G MemorySwapMax=0` (set with `systemctl --user set-property`). A runaway run is killed instead of taking the machine (dc76fd8).

Stable too

No nightly-only API in the crate so the cluster's stable rustc builds it (0c7cd92); `nc.sh` uses nightly locally.

Pedantic allowances

Crate-level `#![allow(clippy::many_single_char_names)]` (treap code); the rest are local `#[allow]`s with a reason.

scripts/*.sh · src/lib.rs:28-32 · rustfmt.toml

What the numbers say

```
$ scripts/run.sh ./target/release/examples/typing \
      10000 --edits 2 --no-check
node record 128 B, use edge 24 B
== 10000 lines (24 threads)
interpreter 85.083ms  engine load+cold 355.172ms (4.17x)
load 189.017ms  cold 166.155ms  runs 11256
stops 1810  gated 4057  pending waits 2033
live 13296  critical path 17  rss 35 MB
edit             median     run   rebnd  waits
insert char      455.981µs    4      1      0
change far def     7.694ms  908      1      0
edit in claim    254.700µs    0      0      0
move block close 267.685µs    2      1      0
```

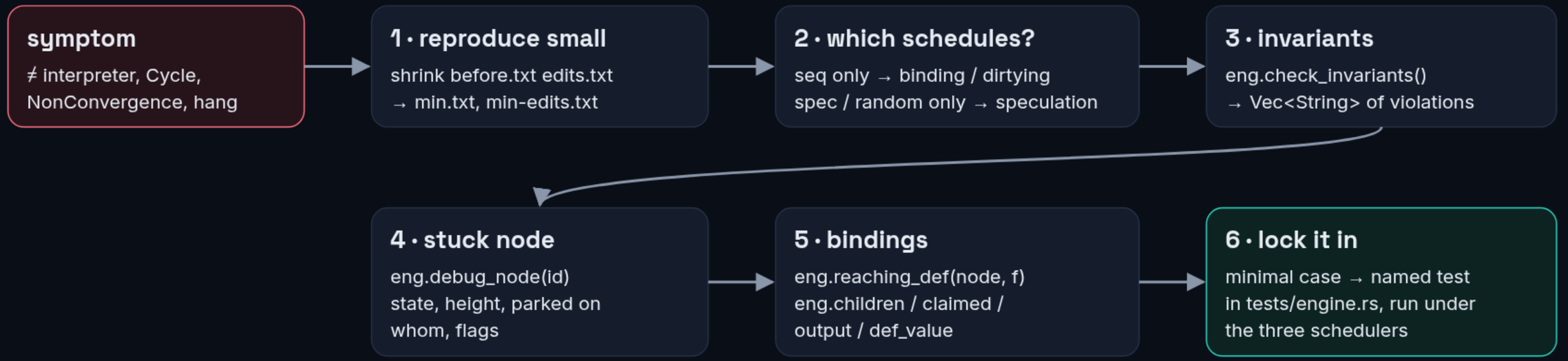
Run while writing this deck (working tree at 191d61f).

Field	Canonical?	Read it as
`nodes_run`	yes	nodes whose inputs changed: the incremental work
`cutoffs`	yes	of those, unchanged outputs and defs: wasted-but-cheap
`rebinds`	yes	uses whose reaching definition changed
`created` / `freed`	yes	unfolding churn
`pending_sets(_defining)`	yes	dynamic def sets resolved (that defined anything)
`runs` / `stops`	no	executions; stopped at a missing value: restarts
`pending_waits`	no	waits on pending structure: rule II at work
`gated`	no	held back by read hints before starting
`speculations` / `confirmations` / `mispredictions`	no	spec kept / confirmed / refuted; storm = mispredictions ≫ confirmations
`steps`	no	batches; ≈ levels; bounds parallelism
`runs_by_op` / `waits_by_op`	mixed	which operation runs or blocks
`critical_path()`	-	height reached: the dependency depth

src/engine.rs Report (≈350-421): "The first group is canonical … the second describes how the work was scheduled"

WORKFLOW · DEBUGGING

When a result is wrong



check_invariants verifies: every local definition indexed at its current depth; every settled use bound where a fresh walk binds it and indexed among its site's users; every decided pending effect reflected in the order. A Cycle lists the parked NodeIds; a cycle that is really a missed wake (418470d) looks the same.

What a commit in this repo looks like

```
Two-phase apply: child heights in the parallel phase; runs reuse th
                                        ← subject: the mechanisms, ';'

A run's result now carries the starting height of each child it
will unfold (...); the commit adds in-text siblings and binds
without walking (bind_fresh). The gate's walks are handed to the
run, so use_field does not walk the same field again in the same
job.                                    ← body: what changed and why

typing 100k cold: 1 thread 3.16s → 2.72s (the run side: gate
walks were 23% of cold and re-walked by use_field); 16 threads
1.43s → 1.37s. The sequential apply itself barely moved
(998 → 984 ms at 16 threads): what remains in it is index
mutation (...), not walks.          ← numbers, before → after
```

One concern per commit, and the honest negative: "the sequential apply itself barely moved".

Numbers from the same benchmark and size, before and after, with the thread count.

"Fixes found on the way" list each bug the fuzz or the client surfaced, each with a test.

Before committing: fmt; clippy pedantic in the three configs; `scripts/tests.sh`; typing with checks at the size you touched.

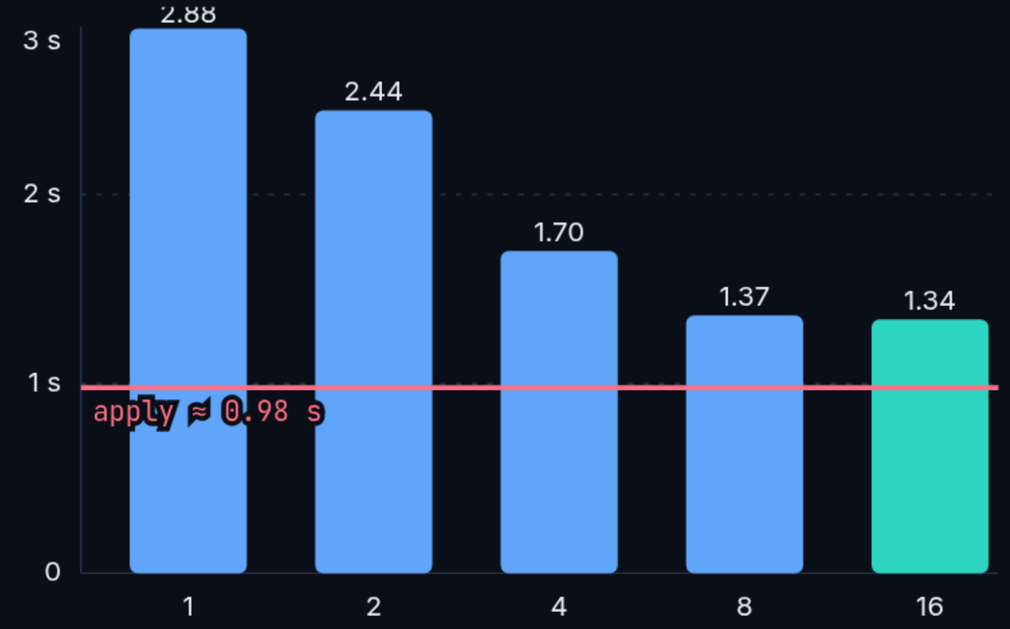
git show de5d2a3 (body abridged)

NUMBERS · 100K LINES

Cold evaluation at 100k lines

Measure	Value	Source
interpreter (sequential reference)	1.00 s	run, this deck
load (build graph from text)	1.72-1.96 s	run (16 t / 1 t)
cold, 1 thread	2.88 s	run · 2.72 s in de5d2a3
cold, 16 threads	1.34 s	run · 1.37 s in de5d2a3
sequential apply at 16 threads	984 ms	de5d2a3
runs · steps · critical path	109966 · 422 · 44	run
rss at 16 threads	281 MB	run
node record · use edge	128 B · 24 B	run (record_sizes)
scope-kind assignment	229 → 13 ms	dc76fd8
bracket edit p99 (insert char)	3.2 → 1.6 s	897e7ca
chain, 1000 links: cold · reclaim edit	281→156 · 155→21 ms	9b7fc36
chain, 4000 links: cold, 1 thread	0.75 s	9b7fc36
replay insert_after	2.5 s (1M) → 4 ms (100k)	1d2633d

cold time vs threads (run: typing 100000 --scaling --edits 0 --no-check)

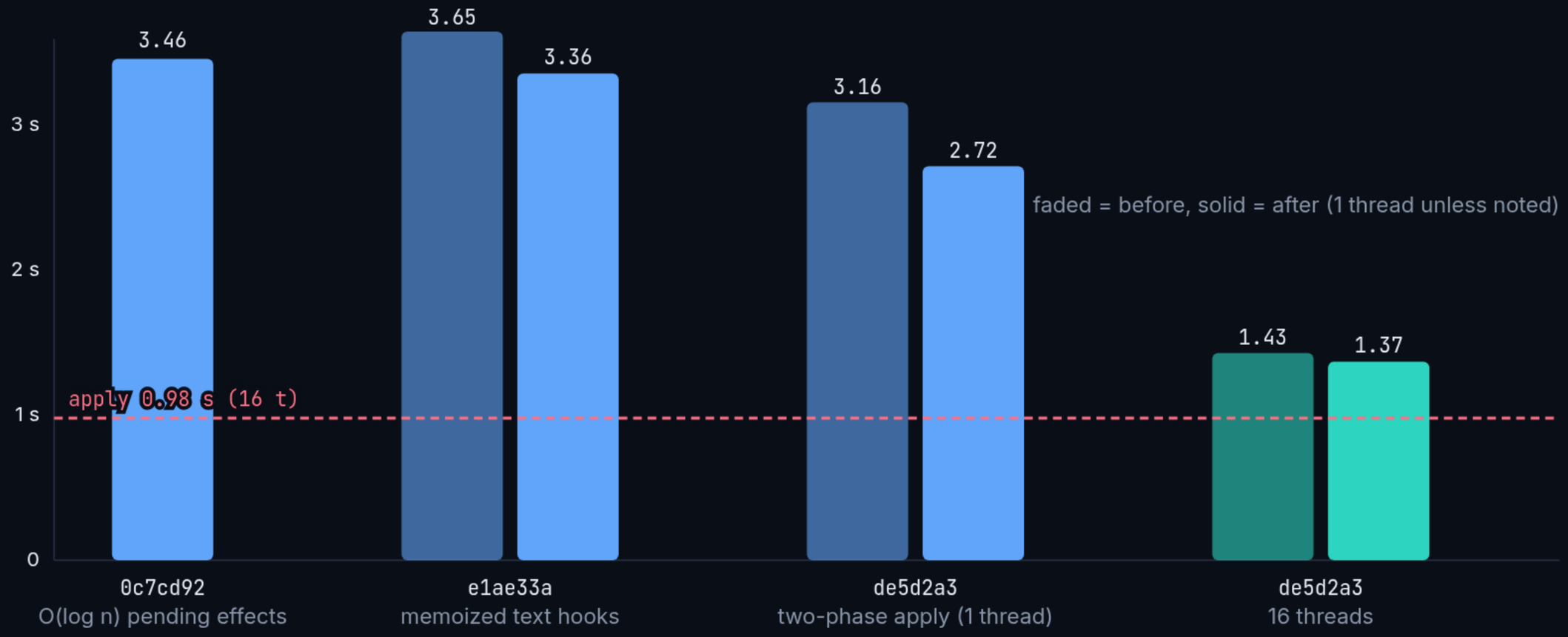


Speedup 2.16x at 16 threads; the apply floor (red) is what bounds it. Interpreter ≈ 1.0 s for comparison.

runs on the dev machine (24 hw threads) at working tree 191d61f; commit numbers as cited

NUMBERS · HISTORY

How cold time at 100k moved, commit by commit



git log: 0c7cd92, e1ae33a, de5d2a3 (typing 100k cold)

OPEN PROBLEMS

Where to look next

1



Sequential apply

~0.98 s of 1.34 s cold at 16 threads. What remains is **index mutation**: order-tree inserts, `local_ix`/`global_ix`/`defs_ix`/`users` updates (de5d2a3). Ideas need to keep the commit order deterministic (results sorted by `NodeId`).

look at: apply, set_children, set_uses, add_def, bind_fresh

2



mispredict, re-run, mispredict

Speculation storms on pending-claim chains

Each link's extent depends on the previous claim's result, so a chain resolves one claim per step. The symptom to look for is mispredictions growing with the chain length when speculation is on (not measured for this deck). `chains.rs` measures it (`SPEC=off|predicted|all`).

look at: recompute_claim, refute_assumers, Report.mispredictions

3



Deep-unfold traces start nodes too early

Reported for traces with deep unfolding: nodes are started many times before their inputs are ready (each start ends in a `Stop`; watch `stops` vs `runs`). Not reproduced for this deck. Child heights from read hints (`child_heights`) help only where the trace gives hints.

look at: child_heights, gate_walks, End::Raise, Report.stops / gated

de5d2a3 · 9b7fc36 · examples/chains.rs · examples/replay.rs

GETTING STARTED

A first week

Day 1 · read the model

`src/lib.rs, engine.rs:1-32, the Operation trait, Ctx::use_field`

Day 1 · run the tour

```
scripts/nc.sh build --release --examples ; scripts/run.sh
./target/release/examples/lang
```

Day 2 · follow one batch

`drain → run_job → apply → finish, with scripts/profile.sh 1000`

Day 3 · binding

```
walk, walk_effects; break it on purpose and watch scripts/tests.sh
fail
```

Day 4 · a toy feature

`add a keyword in common/mod.rs and its mirror in interp.rs; a test`

Day 5 · measure

`typing 10000 with checks; fuzz; before → after numbers in the commit`

Keep the four rules

If a change makes a result depend on the schedule, the random scheduler and the fuzz will find it. If a change waits on something the binding does not cross, `pending_waits` will show it.

Ask the oracle

When in doubt about semantics, the interpreter is the definition; the engine must match it bytewise.

Press T

for this deck's contents; S for speaker notes.